Diya Shetty NNM24IS081

NNM24IS081

Plag Report

5B and UID

Nitte University

Document Details

Submission ID

trn:oid:::1:3690192979

Submission Date

Oct 25, 2026, 9:07 PM GMT+5:30

Download Date

Oct 25, 2026, 9:16 PM GMT+5:30

File Name

OSTask_081_.pdf

File Size

405.6 KB

15 Pages

2,982 Words

16,080 Characters

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

9 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

2 Missing Citation 1%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
2% Submitted works (Student Papers)

Match Groups

- **9** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **2** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	github.com 2%
2	Internet	digital.lib.usu.edu <1%
3	Student papers	University of Nebraska at Omaha <1%
4	Internet	unanswered.io <1%
5	Internet	www.iaeng.org <1%
6	Internet	download.bibis.ir <1%
7	Internet	web.xidian.edu.cn <1%

NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS PROGRAMMING TASK REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem

SUBMITTED BY

Name: Diya Shetty

USN: NNM24IS081

Course: Operating Systems

GitHub Repository: https://github.com/diyashetty24/AI-Assisted-Synchronization-

SUBMITTED TO

Dr Jason Elroy Martis

Associate Professor

Department of Information Science and Engineering, Nitte

1. Problem Statement and Objective

A wrong order of accessing memory from multiple threads may cause a race condition and result in incorrect output. This assignment implements two classic problems of concurrent programming, namely the Readers-Writers problem and the Dining Philosophers problem, solving them with both semaphores and monitors. Thus, four separate programs are created:

- Readers-Writers with a mutex and a semaphore (readers-preference);
- Readers-Writers with a monitor (writer-preference);
- Dining Philosophers with four philosophers and four chopsticks using semaphores;
- Dining Philosophers with four philosophers and four chopsticks using a monitor.

The goal of these programs is to show that threads are capable of ensuring mutual exclusion, synchronization, deadlock prevention and starvation prevention. The programs record all the events that happen to threads while executing them and use the information for building an HTML page that represents the log. As the scheduler controls the exact order of thread execution, two separate runs of the same program will give different results. Semaphores were proposed by Dijkstra [6], monitors by Hoare [8], and the Readers-Writers problem by Courtois et al. [7]. All four programs use the Python standard threading module [1].

Each program logs the events that happen to threads, including waiting, acquiring, using and releasing resources. Also, all programs use the same event recorder and HTML template, which are discussed below.

2. Readers-Writers - Semaphore Solution

The shared datum is an integer called shared_value. Readers execute only reading operations, while writers increment the value. The access to the shared data is protected using mutex_lock for reader_count and a semaphore rw_lock (with initial value of 1), which controls access to the shared data [1].

Reader entry and exit

A reader gets the reader mutex, increasing reader_count by one. In case the reader is the first one in line, it gets rw_lock to prevent the entrance of a writer until the end of the readers' section. On exit, the reader gets the mutex again and decreases reader_count. When the exiting reader is the last one, it releases rw_lock, allowing writers to move further.

Behavior of a writer and remarks

A writer should get rw_lock before writing a new value into shared_value and release it afterwards. Thus, until the writer is within the critical section, no reader or writer can move ahead. Such a configuration is known as the first Readers-Writers solution [7], realized using semaphores [6], and readers are preferred: in case the readers arrive continually, reader_count will be positive constantly, leaving writers wait forever [2].

The writer changes the value in two steps: it gets the current (old) value, waits for some time (random interval) and writes old + 1. Two writers entering the system at the same time may read the same old value, losing the update. So, the last condition, shared_value == NUM_WRITERS_ROUNDS, serves as a very simple mutual exclusion condition. The execution gave "Final value = 6 (expected 6) -> OK." As the first reader gets rw_lock holding the mutex, all the next readers become queued at the mutex.

3. Readers-Writers - Monitor Solution

The monitor is implemented by the ReadWriteMonitor class. It owns a threading.Condition and maintains active_readers, writer_active, waiting_writers, and the shared value.

start_read()

The monitor is implemented using the ReadWriteMonitor class. It owns a threading.Condition and active_readers, writer_active, waiting_writers, and the shared value

end_read()

A reader waits for a writer to be active or for any writer to be waiting. When both are clear, it increments active_readers.

start_write()

A writer increments waiting_writers and waits while another writer or any reader is active. When it can proceed, it decrements waiting_writers and sets writer_active.

end_write()

The writer increments the shared value, clears writer_active, and calls notify_all() to wake waiting readers and writers.
Because new readers also wait whenever waiting_writers is greater than zero, this design prefers writers and avoids a constant stream of readers preventing a writer. Each monitor method runs while holding the condition's lock, so only one thread at a time can examine or modify the shared state [8]. A blocked thread calls wait(), which releases the lock while the thread sleeps. After being awakened, it checks the condition again in a while loop because another thread may have changed the state before it reacquires the lock.
The monitor keeps the shared data and its entry/exit rules together in one class, rather than spreading them across the reader and writer functions. Waiting threads sleep on the condition variable instead of repeatedly polling the state.

4. Dining Philosophers (Four) - Semaphore Solution

The system has 4 philosophers (P0-P3) and 4 chopsticks (0-3). Each chopstick is represented by a Semaphore initialized to 1, indicating that only one philosopher can hold it at a time. Philosopher i uses chopstick i as the left chopstick and chopstick (i + 1) % 4 as the right. Each philosopher repeats the cycle of thinking, becoming hungry, acquiring chopsticks, eating, and releasing chopsticks for three meals.

An additional semaphore named room is initialized to NUM − 1 = 3. A hungry philosopher must acquire room before attempting to take either chopstick and releases room after putting both chopsticks down. This means that at most, three philosophers can compete for chopsticks at once.

3

Without room, all four philosophers could pick up their left chopstick at the same time and wait indefinitely for the right, leading to circular wait and deadlock. By limiting the number of competing philosophers to three, the cycle is broken, and at least one philosopher will be able to obtain both chopsticks, eat, and release them. Starvation is not ruled out entirely, however, because Python's Semaphore does not guarantee FIFO wake-up order.

The program records each chopstick acquisition immediately after it succeeds and each release immediately before releasing the semaphore. This ensures that the event log is in sync with the actual chopstick ownership and that the HTML replay can display the holder accurately.

5. Dining Philosophers (Four) - Monitor Solution

The monitor class is Table. It has a threading.Condition, a four-element boolean list called chopstick_free, and a count of meals consumed by each philosopher.

pick_up(i) and put_down(i)

In pick_up(i), a philosopher indicates that it is hungry and waits in a wait loop until both of its left and right chopsticks are available. Then, it acquires both together. In put_down(i), a philosopher marks both chopsticks as free, records the meal, and calls notify_all() to allow waiting neighbours to check again.

Because the availability checks and acquisition of the chopsticks happen together, while holding the lock of the monitor, a philosopher can never be waiting for one chopstick, holding the other. This eliminates the hold-and-wait condition and avoids deadlock in this implementation. Unlike the semaphore solution, the monitor does not require the additional room semaphore, as it allocates both chopsticks within one critical section. Starvation is still theoretically possible because the monitor does not enforce a fair queue

6. Event Recording and HTML Generation

All four programs make use of the Recorder class in simlib.py. While running, the threads call recorder.state(...), recorder.chopstick(...) and recorder.var(...) every time any of the threads changes its state, acquires or releases a chopstick, or when any shared variable is accessed. All events are stored in a list along with the time they occurred, an event type and a message describing the event. The events are also printed on the console for the user to see the progress of the simulation.

The events are stored in a list protected by a threading.Lock inside the Recorder class so that not two threads can write to it concurrently. Once the execution finishes, the write_html() function is called which writes the events into the template.html file as JSON data. Then, the JavaScript code running on the page parses the JSON and rebuilds the log. The controls at the top allow the user to Start, Pause or Reset the simulation, change the speed of the animation with a range input or move the timeline slider to seek somewhere in the log. The right panel displays all the logged events in chronological order.

For Readers-Writers, the HTML page displays all the readers and writers as boxes with a color indicating their current state. The color changes depending on whether they are idle (grey), waiting to read (yellow), reading (blue) or writing (red). The shared value is also displayed along with the count of active readers.

The page displays the Dining Philosophers as a circular table with four seats. Each philosopher is represented as a box with the same color coding as the Readers-Writers page. The chopsticks are displayed as lines between the philosophers. Since the events are replayed as a log recorded from an actual run of the program, each run may have different events.

How to run

1 `py dining_philosophers_semaphore.py -> dp_semaphore.html`

`py dining_philosophers_monitor.py -> dp_monitor.html`

1 `py readers_writers_semaphore.py -> rw_semaphore.html`

`py readers_writers_monitor.py -> rw_monitor.html`

The Readers-Writers programs optionally take the number of readers, number of writers, and a delay factor (defaults: 4, 2, and 0.3) as arguments. The Dining Philosophers programs optionally take a delay factor.

7. Execution Traces

7.1 Readers-Writers Using Semaphore

In the sample run, W2 writes first while R1-R4 (readers) and W1 (writer) wait. When both writes are done, all of the four readers read concurrently. Summary: 4 readers, 2 writers, 3 rounds each, final value 6 (expected 6).

Example event sequence:

```
0.297s - W2 is now waiting
0.297s - W2 is writing
0.307s - W1 is now waiting
0.320s - R2 is now waiting
0.367s - R3 is now waiting
0.376s - R1 is now waiting
0.405s - R4 is now waiting
0.674s - W2 finished writing
0.675s - W1 is writing
1.091s - W1 finished writing; R1-R4 read value 2
```

The exact order varies between runs because thread scheduling is concurrent and random delays are used.

7.2 Readers-Writers Using Monitor

The monitor trace also contains exclusive writing. After W1 completes, the waiting readers are notified and they read the shared value. The example run ends with the value 6, as expected. Each waiting thread rechecks the condition after being notified by notify_all().

For instance, 0.206s - W2 is waiting and writes, 0.215s - W1 is waiting, 0.244s - 0.427s - readers are waiting, 0.492s - W2 ends, 0.493s - W1 writes, 0.665s - W1 ends, and the readers begin reading.

7.3 Dining Philosophers Using Semaphore

Philosophers P3, P0, and P2 pickup the chopsticks and eat sequentially or concurrently without having to share a common chopstick. Each philosopher has three meals and the program completes successfully without deadlock.
For example, 0.208s - P3 is hungry takes the chopsticks 3 and 0 and begins to eat; 0.367s - P3 finishes and puts them down and P0 picks up the 0 and 1; 0.381s - P2 picks up 2 and 3. The room semaphore, which is initially 3, prevents all four philosophers from entering the chopstick-acquiring stage at the same time.

7.4 Dining Philosophers Using Monitor

The monitor takes both chopsticks in one stroke, as indicated by the timestamps. Every philosopher dines 3 times, and the program never deadlocks.
Sample events: 0.153s - P0 takes up chopsticks 0 and 1 and eats; 0.158s - P2 takes up 2 and 3 and eats; 0.359s - P0 lowers down and releases both chopsticks; 0.370s - P2 does the same. The sample run has 2 consecutive “P0 is now thinking” messages because put_down() stores the state and the loop logs it again at the beginning of the next cycle.

8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race condition

A race condition happens when threads operate on shared data that isn't properly synchronized and the outcome depends on the timing of these operations. This is why in Readers-Writers both writers can read the same old value and increment it by one, thereby losing one of the increments. This error is prevented in both cases and can be checked with the final value.

Mutual exclusion

Only one writer can modify the shared value at the same time, and readers cannot access it at the same time as a writer. The semaphore solution uses rw_lock, while the monitor uses its lock together with writer_active and active_readers. In Dining Philosophers, only one philosopher can hold a chopstick at a time, which is enforced by Semaphore(1) or the chopstick_free flags.

Deadlock

Deadlock requires that mutual exclusion, hold-and-wait, no preemption, and circular wait, occur simultaneously [9]. The semaphore solution breaks the circular wait by allowing only three competing philosophers with room = 3. The monitor solution removes hold-and-wait by allowing a philosopher to take both chopsticks at once or neither.

Starvation

Starvation happens when a thread waits too long because other threads get served first. The semaphore Readers-Writers solution is biased towards readers. Hence, there may be starvation of writers if readers keep arriving [4]. The monitor solution is biased towards writers because new readers block waiting for a writer to finish; a constant stream of writers can in turn starve readers. Starvation is still possible in both implementations of Dining Philosophers because Python's Semaphore and Condition objects do not guarantee strictly FIFO wake up order

9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Structure	Separate primitives (Lock, Semaphore) and shared variables used by the code.	One class contains shared data, a lock, and a condition variable.
Waiting	Threads block inside acquire().	Threads sleep in condition.wait() until notified.
Resource control	The programmer calls acquire() and release() at the correct points in each thread.	Access rules are encapsulated in monitor methods such as start_read() and pick_up().
Error risk	Missing or misordered acquire()/release() calls can cause races or deadlock.	Encapsulation reduces mistakes; a key risk is using if instead of while for condition checks.
Abstraction	Lower-level synchronization mechanism.	Higher-level abstraction.
Conditions	Expressed through counters and semaphore values such as reader_count and room.	Expressed as boolean checks on the monitor's state.

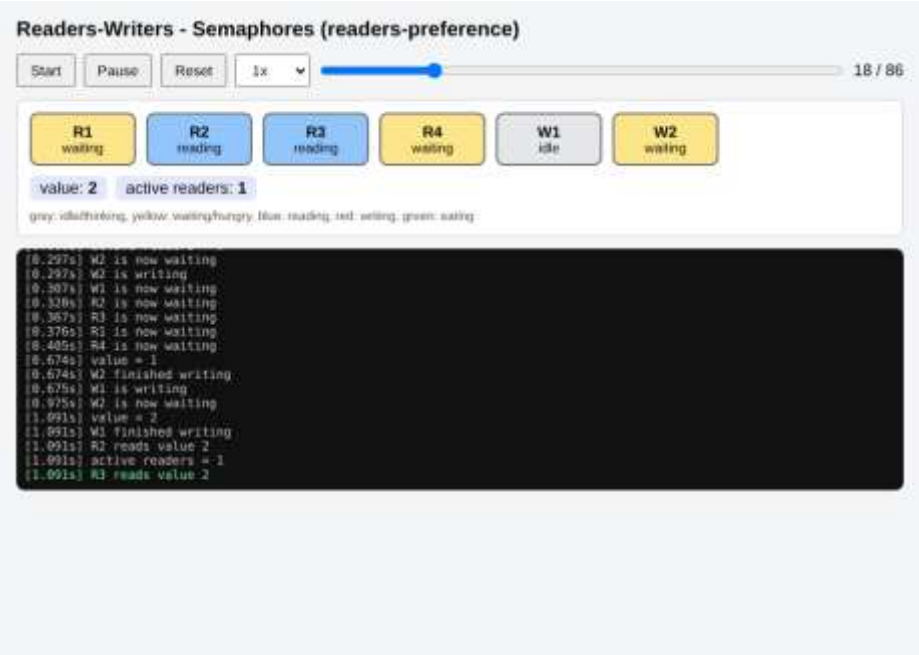
Conclusion:

Semaphores are versatile, but for each acquire, the programmer must correctly match a release. A monitor groups shared data, locks and conditions, thus making their accessibility and modification rules easier to verify [8] [10].

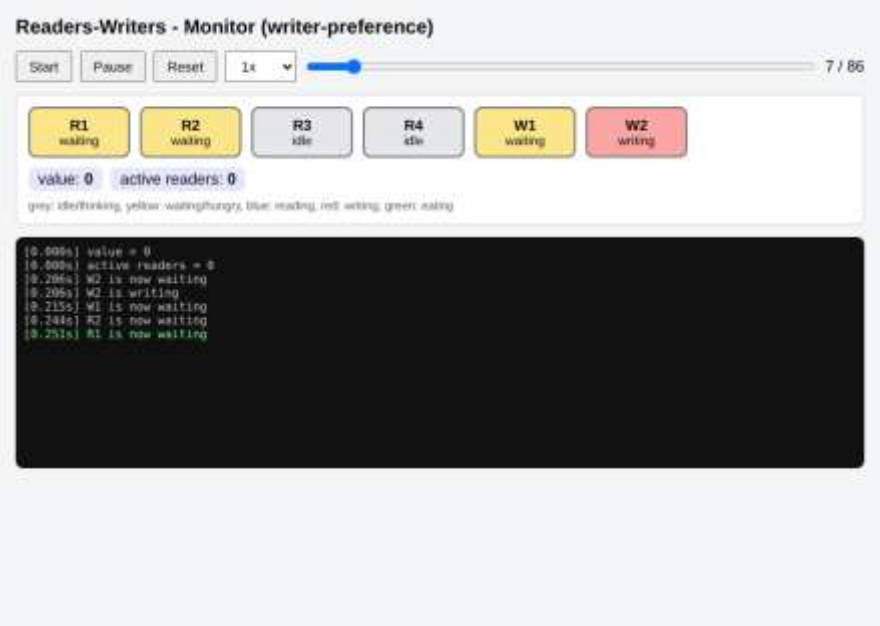
10. Screenshots of Generated HTML Simulations

The following screenshots were taken from the original report and show the HTML simulations at points during replay

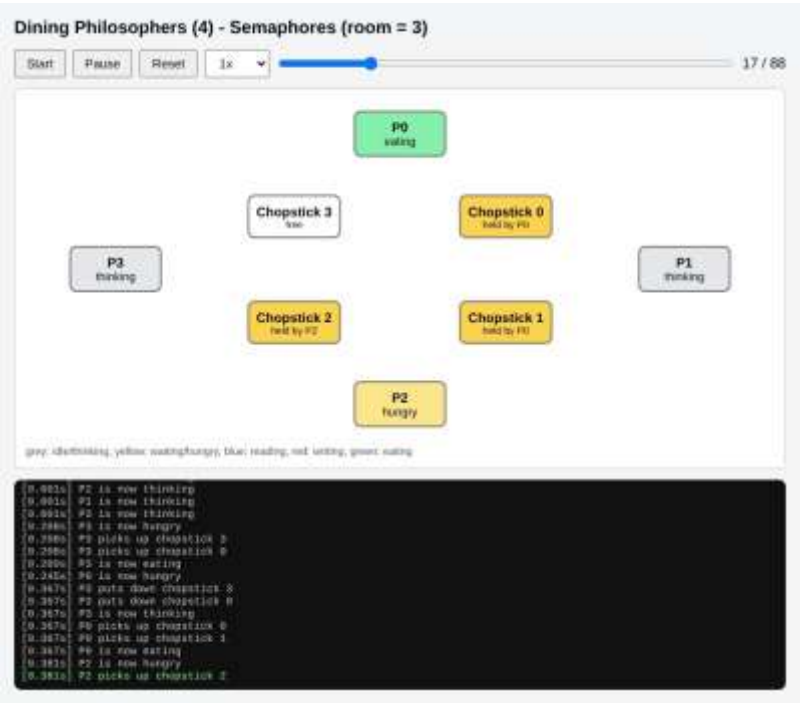
10.1 Readers-Writers Using Semaphore



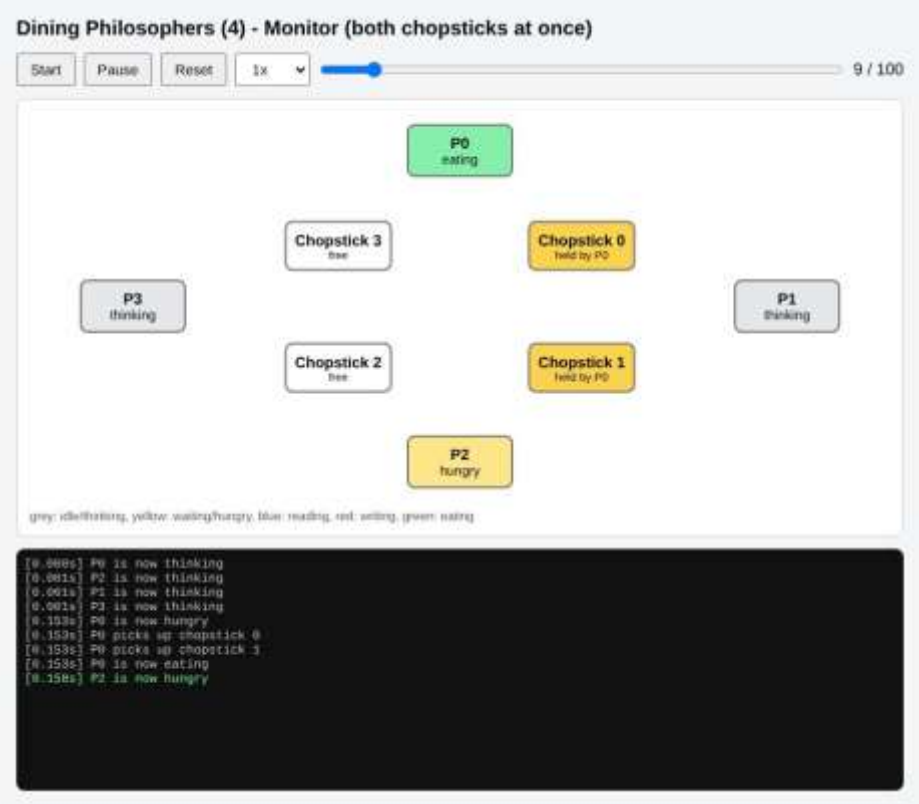
10.2 Readers-Writers Using Monitor



10.3 Dining Philosophers Using Semaphore



10.4 Dining Philosophers Using Monitor



11. AI Usage Summary

Review and limitations

The assistance of an AI was used during the preparation of this report for wording, organisation and document presentation. Technical discussions, code behaviour, references, and test claims should be reviewed against the programs and output actually generated. The report is not asserting that the AI performed or tested the programs it describes.

Representative prompts matching this editing work

The prompts below summarize the assistance used; they are representative prompts, not a verbatim chat transcript.
Prompt 1: “Use the supplied humanized document to update the operating-systems assignment report while retaining the existing technical sections and simulation screenshots.”
Prompt 2: “Improve the report’s wording, organisation, heading hierarchy, and formatting without changing the intended meaning of the synchronization explanations.”
Prompt 3: “Format the Semaphore vs Monitor comparison as a readable table and keep the simulation screenshots with clear captions.”
Prompt 4: “Check the document layout so major section headings stay with their content, add consistent page numbering, and improve alignment.”
ChatGPT was used to help revise and organise the supplied document, improve wording and consistency, and format the section headings, comparison table, screenshots, and page layout. The student should review the final report against the code and outputs actually produced.

AI assistance used

AI-assisted editing and formatting. The student is responsible for verifying the accuracy of the technical content and for disclosing AI use according to the course requirements.

12. Reference:

1] Python Software Foundation, “threading - Thread-based parallelism,” Python 3 Documentation. https://docs.python.org/3/library/threading.html

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts.

[3] A. S. Tanenbaum and H. Bos, Modern Operating Systems.

[4] A. B. Downey, The Little Book of Semaphores. https://greenteapress.com/wp/semaphores/

[5] Anthropic, “Claude,” AI assistant.

[6] E. W. Dijkstra, “Cooperating sequential processes,” Technological University Eindhoven, EWD123, 1965.

[7] P. J. Courtois, F. Heymans, and D. L. Parnas, “Concurrent control with ‘readers’ and ‘writers’,” Communications of the ACM, vol. 14, no. 10, pp. 667-668, 1971.

[8] C. A. R. Hoare, “Monitors: An operating system structuring concept,” Communications of the ACM, vol. 17, no. 10, pp. 549-557, 1974.

[9] E. G. Coffman, M. J. Elphick, and A. Shoshani, “System deadlocks,” ACM Computing Surveys, vol. 3, no. 2, pp. 67-78, 1971.

[10] W. Stallings, Operating Systems: Internals and Design Principles, persons

13. Academic Integrity Declaration

I declare that I have tested and verified the correctness of the provided programs myself. I have disclosed the AI tools and external sources of information that were used while performing this assignment. Moreover, I can explain the synchronization logic, HTML code simulation, and changes that have been made to the code. Furthermore, I have not presented someone else’s work as my own.

Name: Diya Shetty

USN: NNM24IS081

Signature: _____

Date: _____